

fancyhdrvarioref



Szoftver projekt laboratórium

8. ÖSSZESÍTETT DOKUMENTUM

Csapat

49 - komenymag

Konzulens

Marcsingó Ádám

Csapattagok

Görgényi András Levente	GSQEM0	andrisgorgenyi@gmail.com
Dulcz Bence	Y9LPT9	dulcz.bence@gmail.com
Varga Gergely	IBXDS0	vargageri05@gmail.com
Nagy Patrik	C3IJYQ	nagy.patrik0511@gmail.com
Tóth Dávid Pál	H9JYWA	tothdavid476@gmail.com

2026. május 10.

11. fejezet

Grafikus felület specifikációja

11.1. A grafikus interfész

A menürendszer, a kezelői felület grafikus képe. A grafikus felület megjelenését, a használt ikonokat, stb screenshot-szerű képekkel kell bemutatni. Az építészetben ez a homlokzati terv.



11.1. ábra. Példa kép

11.2. A grafikus rendszer architektúrája

A felület működésének elve, a grafikus rendszer architektúrája (struktúra diagramok). A struktúra diagramokon a prototípus azon és csak azon osztályainak is szerepelnie kell, amelyekhez a grafikus felületet létrehozó osztályok kapcsolódnak.

11.2.1. A felület működési elve

A grafikus felület a meglévő üzleti logikát (motor, jarmuvek, játékosok, takarítófejek, terkep csomagok) változtatás nélkül használja fel — a modell rétegben a GUI miatt csak additív bővítések történtek (új interfész, új getterek, új mezők), egyetlen meglévő metódus viselkedése sem módosult. Így a korábbi parancssoros prototípus ugyanúgy működik, mint korábban.

A három réteg felelőssége

A felület tiszta MVC elrendezést követ, három különálló csomagra bontva:

- Modell (motor, jarmuvek, játékosok, takarítófejek, terkep) — az állapot és a játékszabályok.
- Nézet (gui.view) — a Swing-panelek (MapPanel, HudPanel, ContextPanel, EventLogPanel), amelyek a modell aktuális állapotát rajzolják. A nézet csak olvas a modellből, soha, sehol nem módosítja közvetlenül.

- Vezérlő (`gui.controller`) — a `MapController` és az `ActionController`. Ezek a felhasználói inputot fordítják le model-hívásokká (pl. `Hokotro.takaritSav`, `Jarmu.moveToNode`, `GameActions.endRound`).

A megjelenítés alapelve: kevert (push + pull)

Push oldal — eseményalapú értesítés. A motor csomagba felvettünk egy `GameStateListener` interfészt egyetlen `onStateChanged(GameState state)` metódussal. A `GameState` egy `addListener` / `fireStateChanged` páros segítségével tartja számon a feliratkozott listenereket. Amikor a játékállapot ténylegesen változik (jármű kijelölése, lépés, takarítás, kör vége, vásárlás), egy magasabb szintű művelet végén — jellemzően az `ActionController`-ben vagy a `GameActions.endRound`-ban — a vezérlő meghívja a `state.fireStateChanged()`-et. A `GameWindow` minden view-panelt feliratkoztat listenerként, tehát az értesítés minden megjelenítendő komponensnek kézbesítődik.

Pull oldal — közvetlen lekérdezés rajzoláskor. A view-panelek `onStateChanged` implementációja nem ad át adatot magával az eseménnyel; egyszerűen kiváltja a saját frissítését (`repaint()` a `MapPanel`-en, vagy a label-ek és gombok újraépítése a `ContextPanel`-ben). A tényleges állapotot a panel a saját rajzolási vagy frissítési ciklusában közvetlenül olvassa le a modellből: a `MapPanel.paintComponent` például a `state.getAllUtak()`, `state.getAllEntities()`, `Sav.ho`, `Sav.ice` értékeit kérdezi le, a `ContextPanel` pedig a kiválasztott entitást a `state.selected()`-en keresztül. Így a megjelenítés mindig az aktuális, friss állapotot tükrözi anélkül, hogy a model-eseménynek bármilyen tartalmat kéne hordoznia.

11.2.2. A felület osztály-struktúrája

Osztálydiagram. Minden új osztály, és azon régiek, akik az újakhoz közvetlenül kapcsolódnak.



11.2. ábra. Példa kép

11.3. A grafikus objektumok felsorolása

Az új osztályok felsorolása. Az régi osztályok közül azoknak a felsorolása, ahol változás volt. Ezek esetén csak a változásokat kell leírni.

11.3.1. Új osztályok

11.3.1.1. `GameStateListener`

■ Felelősség

Push-event interfész a model → view kommunikáció megvalósításához. Lehetővé teszi, hogy a modell értesítse a feliratkozott nézeteket (view-kat) az állapotváltozásról, így

azok frissíthetik a megjelenített adatokat.

- Interfészek

-

- Össosztály

-

- Attribútumok

-

- Metódusok

- ◇ *+onStateChanged()* : *void* - A modell hívja meg, amikor az állapota megváltozik, jelezve a nézetnek a grafikus frissítés szükségességét.

11.3.1.2. Main

- Felelősség

A GUI alapú alkalmazás belépési pontja. Feladata a rendszerszintű megjelenés (Look Feel) beállítása és a főmenü elindítása az Event Dispatch Thread-en (EDT).

- Interfészek

-

- Össosztály

-

- Attribútumok

-

- Metódusok

- ◇ *+main(args : String[])* : *void* - A program belépési metódusa.

11.3.1.3. MainMenu

- Felelősség

A játék indítóképernyője. Lehetővé teszi a nehézségi szint és a kezdő hókotrófej kiválasztását, majd elindítja a játéklablakot.

- Interfészek

-

- Össosztály

JFrame

■ Attribútumok

- ◇ *-difficultyCombo* : *JComboBox* - Nehézség választó (Könnyű, Közepes, Nehéz).
- ◇ *-headRadioGroup* : *ButtonGroup* - Rádiógomb csoport a kezdő fej kiválasztásához (Söprő/Jégtörő).
- ◇ *-startButton* : *JButton* - A játék indítását kiváltó gomb.

■ Metódusok

- ◇ *+MainMenu()* - Konstruktor, felépíti a menü grafikus felületét.
- ◇ *-onStartGame()* : *void* - Eseménykezelő, amely létrehozza a GameWindow-t a választott paraméterekkel, majd bezárja a főmenüt.

11.3.1.4. GameWindow

■ Felelősség

A játék fő ablaka, amely összefogja a különböző nézeteket és kontrollereket. Összeállítja a játék kezdeti állapotát és kezeli a panelek elrendezését.

■ Interfészek

-

■ Össosztály

JFrame

■ Attribútumok

- ◇ *-state* : *GameState* - A játék aktuális modell-állapota.
- ◇ *-mapPanel* : *MapPanel* - A térkép megjelenítő panel.
- ◇ *-hudPanel* : *HudPanel* - A felső információs sáv.
- ◇ *-contextPanel* : *ContextPanel* - A jobb oldali interakciós panel.
- ◇ *-eventLogPanel* : *EventLogPanel* - Az alsó eseménynapló panel.

■ Metódusok

- ◇ *+GameWindow(diff : Difficulty, head : HeadType)* - Létrehozza az ablakot, inicializálja a modellt és a nézeteket, majd feliratkoztatja a paneleket a modell eseményeire.

11.3.1.5. GameInitializer

■ Felelősség

Statikus segédosztály, amely a specifikáció szerinti várostérképet és a kezdő objektumokat (csomópontok, utak, járművek, NPC-k) hozza létre a memóriában.

■ Interfészek

-

■ Össosztály

-

■ Attribútumok

-

■ Metódusok

- ◇ *+createCity(state : GameState) : void* - Feltölti a kapott állapotot a hard-coded adatokkal: Telephely, Főtér, Vasútállomás, stb., valamint a játékosok és NPC-k kezdőhelyzeteivel.

11.3.1.6. GameLayout

■ Felelősség

A térkép statikus elhelyezkedéséért felelős segédosztály. Tárolja a koordinátákat és a megjelenítéshez szükséges konstansokat.

■ Interfészek

-

■ Össosztály

-

■ Attribútumok

- ◇ *+NODE_RADIUS : int* - Csomópontokrajzolás sugarasugara.
- ◇ *+LANE_WIDTH : int* - Sávok szélessége pixelben.
- ◇ *-nodePositions : Map<String, Point>* - Csomópont azonosítóhoz rendelt koordináták.
- ◇ *-displayNames : Map<String, String>* - Belső azonosítók magyar megfelelői.

■ Metódusok

- ◇ *+getPosition(nodeId : String) : Point* - Visszaadja egy adott csomópont koordinátáját.
- ◇ *+getDisplayNames(id : String) : String* - Visszaadja az azonosító magyar nevét.

11.3.1.7. MapPanel

■ Felelősség

A város térképének és a rajta lévő egységeknek a vizuális megjelenítése. Kirajzolja a csomópontokat, az utak állapotát és a járművek aktuális pozícióját.

■ Interfészek

GameStateListener

■ Össosztály

JPanel

■ Attribútumok

◇ *-state : GameState* - Referencia a modellre az adatok lekéréséhez.

■ Metódusok

◇ *paintComponent(g : Graphics) : void* - Kirajzolja a térképet: sárga/zöld/szürke csomópontok, utak a színkódolt állapotuk szerint, és a járművek ikonjai.

◇ *+vehicleAt(p : Point) : String* - Visszaadja a megadott koordinátán található jármű nevét, ha nincs ott semmi, null-t.

◇ *+onStateChanged() : void* - A modell jelzésére újrarajzolja a panelt (repaint).

11.3.1.8. HudPanel

■ Felelősség

A játék felső állapotsávja, amely az általános információkat (kör, aktív játékos, vagyon, balesetek) jeleníti meg.

■ Interfészek

GameStateListener

■ Össosztály

JPanel

■ Attribútumok

◇ *-roundLabel : JLabel* - Kör sorszámát mutató felirat.

◇ *-wealthLabel : JLabel* - A játékosok vagyonát mutató felirat.

◇ *-doneButton : JButton* - A kört befejező "Kész vagyok" gomb.

■ Metódusok

◇ *+onStateChanged() : void* - Frissíti a feliratokat és a gomb szövegét az aktív játékos neve alapján.

11.3.1.9. ContextPanel

■ Felelősség

A kiválasztott jármű részleteit és a rajta végezhető akciókat (mozgás, takarítás, telepelyi műveletek) megjelenítő panel.

■ Interfészek

GameStateListener

■ Őosztály

JPanel

■ Attribútumok

- ◇ *-actionContainer* : *JPanel* - Dinamikusan változó gombok tárolója.
- ◇ *-infoArea* : *JTextArea* - Jármű adatainak (üzemanyag, töltet) megjelenítése.

■ Metódusok

- ◇ *+onStateChanged()* : *void* - A kijelölt jármű típusa és helyzete alapján újragenerálja az akciógombokat.

11.3.1.10. EventLogPanel

■ Felelősség

A játék eseményeinek szöveges naplózása és a jelmagyarázat megjelenítése.

■ Interfészek

GameStateListener

■ Őosztály

JPanel

■ Attribútumok

- ◇ *-logArea* : *JTextArea* - Az eseményeket listázó szövegdoz.

■ Metódusok

- ◇ *+onStateChanged()* : *void* - Lekéri a modell legfrissebb eseményeit és megjeleníti azokat.

11.3.1.11. MapController

■ Felelősség

A térképen történő egérekattintások kezelése, a járművek kijelölésének és deszelektálásának vezérlése.

■ Interfészek

MouseListener

■ Őosztály

MouseAdapter

■ Attribútumok

- ◇ *-mapPanel* : *MapPanel* - A megfigyelt térkép panel.
- ◇ *-state* : *GameState* - A modell, ahol a kijelölést állítja.

■ Metódusok

- ◇ *+mouseClicked(e : MouseEvent) : void* - Kattintáskor a vehicleAt segédmetódussal lekérdezi, van-e ott jármű, és frissíti a modellben a kijelölt objektumot.

11.3.1.12. ActionController

■ Felelősség

A felhasználói felületről érkező gombnyomásokat fordítja le konkrét modell-műveletekre, kezeli a körök közötti váltást és a játék végének logikáját.

■ Interfészek

-

■ Ősosztály

-

■ Attribútumok

- ◇ *-state : GameState* - A játék modell-állapota.

■ Metódusok

- ◇ *+onMoveTo(roadId : String) : void* - A kijelölt jármű mozgatása a kiválasztott útra.
- ◇ *+onTakarit() : void* - A takarítási akció elindítása a felszerelt fejnek megfelelően.
- ◇ *+onBuyHokotro() : void* - Új hókotró vásárlása a telephelyen.
- ◇ *+onPlayerDone() : void* - Az aktív játékos körének lezárása, váltás a következő játékosra, vagy NPC körök futtatása.
- ◇ *-ensureControllable() : boolean* - Ellenőrzi, hogy a kijelölt jármű az aktív játékosé-e.
- ◇ *-showGameOver(winner : String) : void* - Megjeleníti a játék vége dialógust a győztesrel.

11.3.2. Megváltozott osztályok

11.3.2.1. Osztály2

■ Felelősség

Felelősség leírása

■ Interfészek

Megvalósított interfészek felsorolása

■ Ősosztály

Ős-Ősosztály → Ősosztály...

■ Attribútumok

◇ $+A [0..*]$ - adattag A

Milyen publikus, protected és privát metódusokkal rendelkezik. Metódusonként precíz leírás, ha szükséges, activity diagram is a metódusban megvalósítandó algoritmusról.

Minden olyan metódusnak szerepelnie kell, amelyiket az osztály megvalósít vagy felüldefiniál.

■ Metódusok

◇ $+B(A\ a) : void$ - metódus B

11.4. Kapcsolat az alkalmazói rendszerrel

Szekvencia-diagramokon ábrázolni kell a grafikus rendszer működését. Konzisztens kell legyen az előző alfejezetekkel. Minden metódus, ami ott szerepel, fel kell tűnjön valamelyik szekvenciában. Minden metódusnak, ami szekvenciában szerepel, szereplnie kell a valamelyik osztálydiagramon.

11.5. Napló

A napló tartalmazza az előző beadás óta eltelt időszak történéseit időrendben. A naplóból egyértelműen ki kell derülnie, hogy az egyes anyagrészeket ki és mennyi idő alatt készítette.

A napló bejegyzésekből áll. Minden bejegyzésnek tartalmaznia kell:

- a történes kezdetének időpontját, nap-óra pontossággal
- történes időtartamát, óra felbontással
- a szereplő(k) nevét (Kérjük a szereplők VEZETÉKNEVÉT használni)
- a tevékenység leírását.

Amennyiben a tevékenységben több szereplő vesz részt, akkor aza tevékenység csak értekezlet lehet, amelynek az eredményei DÖNTÉSEK. A döntéseket precízen meg kell szövegezni (Pl.: Az X objektum Y és Z metódusainak kódját W készíti el Q határidőre). Ha a bejegyzés egyetlen személyhez kötődik, akkor meg kell adni, hogy a tevékenység milyen dologra irányul. A dolog a feladat kapcsán elkészítendő termék, amelynek a (esetleg korábban) beadott anyagban megtalálhatónak kell lenni. A naplóbejegyzés felbontásának egysége szöveges, rajzos anyag esetében az ábra, diagram, vagy kb. fél-egy oldalnyi szöveg. Kódban az egység a metódus. (Pl.: A 3. ábrán látható szekvencia-diagram kidolgozása, vagy az X objektum Y és Z metódusainak kódolása és belövése.

	Kezdet	Időtartam	Résztvevők	Leírás
feb. 18. 16h	1 óra	Csapat	Értekezlet	Döntés: Segédeszközök kiválasztása (git, trello, drive)